

REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND
UNIVERSITY OF BUEA



REPUBLIQUE DE CAMEROUN
PAIX-TRAVAIL-PATRIE
UNIVERSITE DE BUEA

TITLE:

MOBILE APP DEVELOPMENT PROCESS

CEF 440, Internet programming and Mobile Programming

Lecturer: Dr Valery Nkemeni

Report by Group 21

S/N	NAME	Specialisation	Matricule
1	CHE RANDY FANG	Software	FE23A030
2	ECHUNDIP CARL-LAWRENCE N.	Networking	FE23A040
3	MAKANE THANKS MOSAKA	Software	FE23A085
4	NUMFOR PRECIOUS LUM	Software	FE23A126
5	TEMFA NKOMBONG FORTUNEE	Software	FE23A168

Contents

TASK 4: UML DIAGRAMS AND RELATIONSHIPS.....	3
Introduction	3
Importance of the UML Design Phase.....	3
Context Diagram.....	6
Use case Diagram	9
Data flow Diagram.....	13
Sequence Diagram.....	17
Class Diagram.....	24
Deployment Diagram	35
Overall Conclusion.....	40

TASK 4: UML DIAGRAMS AND RELATIONSHIPS

Introduction

This next part of our software development project focuses on system modelling and design using UML (Unified Modelling Language) diagrams. At this stage, the requirements gathered from earlier phases are transformed into a visual representation of how the system will function, how users will interact with it, and how different components of the system relate to one another.

UML diagrams help developers, designers, lecturers, and stakeholders understand the structure and behaviours of the proposed system before actual implementation begins. Instead of explaining the system only with text, UML provides standardized diagrams that visually describe the system architecture, processes, interactions, and data flow.

This phase is very important because it acts as a bridge between system analysis and system implementation. It ensures that the development team has a clear blueprint of the project before coding starts.

Importance of the UML Design Phase

The UML design phase plays several critical roles in software engineering projects:

1. **Improves System Understanding:** UML diagrams provide a clear visual explanation of how the system works. This makes it easier for developers and stakeholders to understand the project requirements and system behaviours.
2. **Enhances Communication:** Diagrams create a common language between developers, supervisors, clients, and team members. Everyone can easily interpret the system design without needing to read complex technical descriptions.
3. **Helps Detect Errors Early:** Designing the system visually helps identify missing requirements, logical mistakes, or weak relationships before implementation begins, reducing future development costs and errors.
4. **Serves as a Blueprint for Development:** UML diagrams guide programmers during implementation by showing:

- System structure

- User interactions

- Database relationships

- Process flows

- Object communication

5. **Improves Documentation:** The diagrams become part of the project documentation and can be used later for:

- System maintenance

- Future upgrades

Training new developers

Academic presentation and defence

6. **Supports Better Planning:** By visualizing components and interactions early, developers can better estimate resources, modules, timelines, and technologies required.

Relationships Used in UML Diagrams

Relationships are connections between system components. Common UML relationships include:

Relationship	Description
Association	General connection between classes
Inheritance	One class derives from another
Aggregation	Weak ownership relationship
Composition	Strong ownership relationship
Dependency	One element depends on another
Include	Mandatory functionality in use cases
Extend	Optional functionality in use cases

UML SPECIFICATIONS:

The UML design phase is one of the most important stages in software development because it transforms system requirements into visual models that guide implementation. UML diagrams improve communication, reduce design errors, and provide a structured blueprint for building the system successfully.

By using diagrams such as Use Case, Class, Sequence, Activity, and Data Flow Diagrams, developers can better understand system structure, interactions, workflows, and relationships, leading to a more organized, efficient, and maintainable software solution.

Abstracts and Diagram Explanations

Context Diagram

This Shows the overall interaction between our Adaptive Mobile E-Learning Platform and external entities such as students, instructors, and administrators. It focuses on system boundaries, data exchange, and major communication flows.

Characteristics: standardized UML, clear actor/component separation, logical flow representation, and support for software analysis and implementation.

Development Approach: derived from the SRS requirements, user stories, functional requirements, and system architecture described in the uploaded project documents.

Use Case Diagram

Represents the functional interactions between users and the platform. It identifies actors and their goals, including login, course access, uploads, and QoE monitoring.

Characteristics: standardized UML, clear actor/component separation, logical flow representation, and support for software analysis and implementation.

Development Approach: derived from the SRS requirements, user stories, functional requirements, and system architecture described in the uploaded project documents.

Deployment Diagram

Illustrates the physical architecture of the system including mobile devices, backend servers, databases, and cloud infrastructure. It explains how software components are distributed across hardware nodes.

Characteristics: standardized UML, clear actor/component separation, logical flow representation, and support for software analysis and implementation.

Development Approach: derived from the SRS requirements, user stories, functional requirements, and system architecture described in the uploaded project documents.

Class Diagram

Describes the static structure of the system by modeling classes, attributes, and relationships. It forms the basis for object-oriented system implementation.

Characteristics: standardized UML, clear actor/component separation, logical flow representation, and support for software analysis and implementation.

Development Approach: derived from the SRS requirements, user stories, functional requirements, and system architecture described in the uploaded project documents.

Data Flow Diagram

Shows how information moves through the platform, including processes, external entities, and data stores. It emphasizes the transformation and storage of data.

Characteristics: standardized UML, clear actor/component separation, logical flow representation, and support for software analysis and implementation.

Development Approach: derived from the SRS requirements, user stories, functional requirements, and system architecture described in the uploaded project documents.

Sequence Diagram

It illustrates the chronological interaction between system components during operations such as login and content access. It focuses on message flow and execution order.

Characteristics: standardized UML, clear actor/component separation, logical flow representation, and support for software analysis and implementation.

Development Approach: derived from the SRS requirements, user stories, functional requirements, and system architecture described in the uploaded project documents.

For our E-learning platform we came up with the following diagrams

Context Diagram

1. Overview

The context diagram provides a high-level view of the Adaptive E-learning Platform and shows how it interacts with external entities. The system is represented as a single central process, while external users and organizations communicate with the system through data flows.

The context diagram focuses mainly on:

- External entities
- Input data to the system
- Output data from the system
- Overall system boundaries

The Adaptive E-learning Platform is placed at the center of the diagram because it is the main system being modeled.

2. External Entities

The following external entities were identified in the context diagram:

a. Student

The student is the primary user of the platform. Students interact with the system to:

- Log into the platform
- Access course materials
- Submit quiz answers
- Participate in live sessions
- Receive notifications and learning content
- Provide QoE feedback

The system returns learning materials, quiz results, alerts, and progress information to students.

b. Instructor

Instructors are responsible for creating and managing learning content. They interact with the platform to:

- Upload course materials
- Schedule live sessions
- Monitor student performance
- Access reports and analytics

The platform provides instructors with engagement reports and student analytics.

c. **System Administrator**

The administrator manages the platform and ensures smooth system operation. The administrator is responsible for:

- Managing user accounts
- Monitoring system activities
- Reviewing logs and reports
- Maintaining platform security

The platform provides system logs and monitoring information to administrators.

d. **ISP (Internet Service Provider)**

The ISP supports internet connectivity and network communication. It contributes bandwidth and network-related metrics used by the adaptive streaming mechanism.

e. **Content Provider**

Content providers supply additional educational resources and external learning materials to the platform.

f. **Government Regulator**

The government regulator ensures that the system complies with educational and digital policies.

3. **Data Flows**

Several data flows exist between the system and the external entities.

Input Data to the System

Examples include:

- Login credentials
- Course uploads
- Quiz answers
- QoE feedback

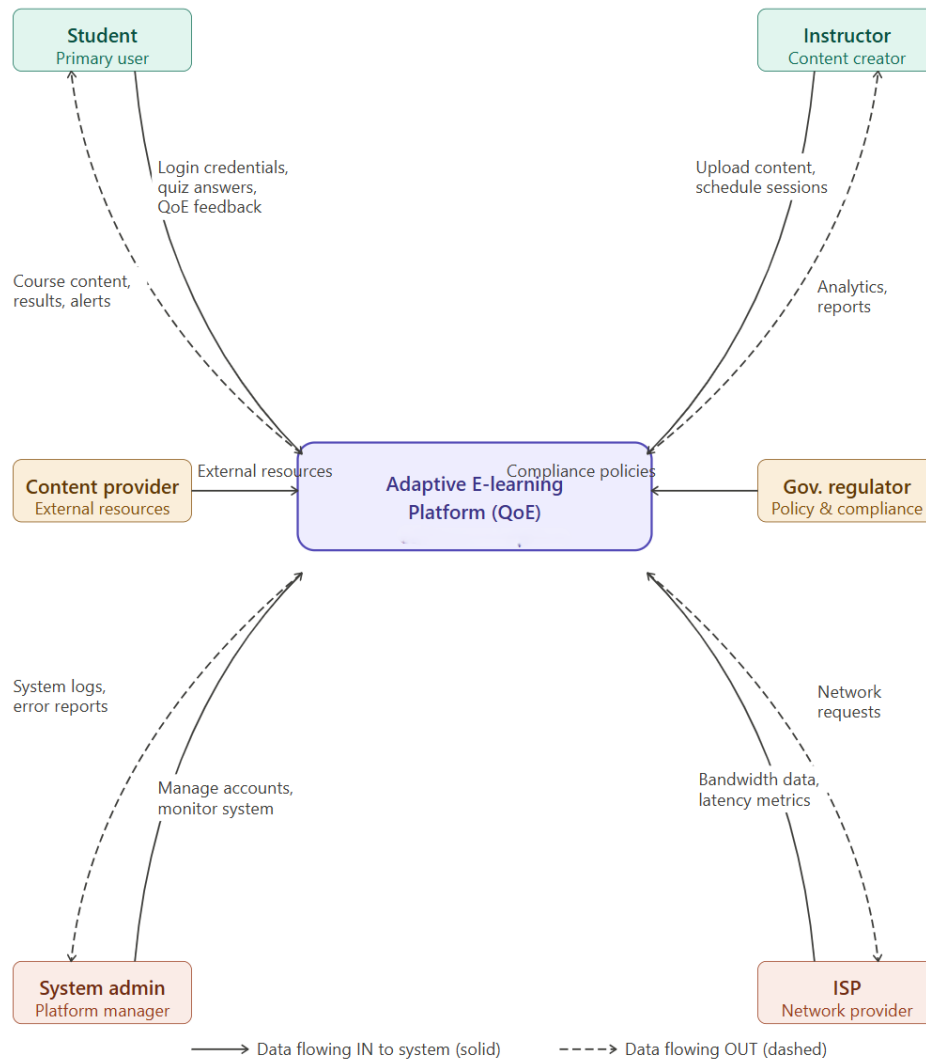
- Lecture schedules
- Network requests
- Administrative commands

Output Data from the System

Examples include:

- Course content
- Quiz results
- Notifications and alerts
- Analytics reports
- Bandwidth statistics
- Error reports and system logs

These data flows demonstrate how the system exchanges information with users and external services.



4. Importance of the Context Diagram

The context diagram is important because it:

- Defines the system boundary clearly.
- Identifies all major external entities.
- Simplifies understanding of the overall system.
- Helps stakeholders understand how the system communicates externally.
- Provides a foundation for detailed system modeling.

The diagram also helps identify external dependencies such as internet services and content providers.

Use case Diagram

1. Overview

The use case diagram illustrates the functional requirements of the Adaptive E-learning Platform and shows how users interact with the system.

The diagram models:

Actors interacting with the system

System functionalities

Relationships between different use cases

The use case diagram focuses mainly on user goals and system behavior.

2. Actors in the System

The major actors identified in the use case diagram are:

a. Student

The student actor represents learners who use the platform to access educational services.

b. Instructor

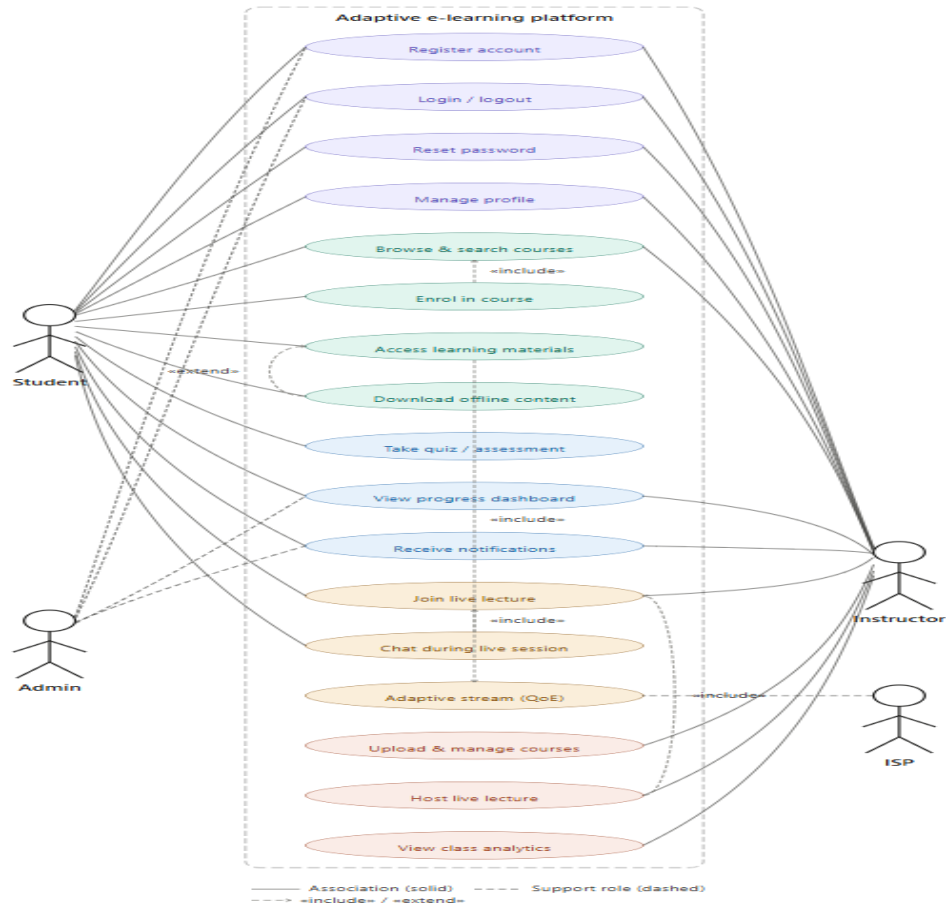
The instructor actor represents lecturers or content creators responsible for course delivery and student monitoring.

c. Administrator

The administrator actor manages the platform, users, and system monitoring activities.

d. ISP

The ISP actor represents the external network provider supporting adaptive streaming and bandwidth communication.



3. Description of Use Cases

The major use cases identified in the system include:

Register Account

This use case allows users to create new accounts using valid credentials.

Login / Logout

This functionality allows authenticated users to access and exit the platform securely.

Reset Password

Users can recover access to their accounts through password reset functionality.

Manage Profile

Users can update personal information and account settings.

Browse and Search Courses

Students can explore available courses and search for learning content.

Enroll in Course

Students can enroll in courses of interest to gain access to learning materials.

Access Learning Materials

Students can access videos, audio lectures, notes, and downloadable resources.

Download Offline Content

This feature allows students to save materials locally for offline learning.

Take Quiz / Assessment

Students can complete quizzes and assessments online.

View Progress Dashboard

Students can monitor completed modules, scores, and learning progress.

Receive Notifications

The system sends notifications about deadlines, announcements, and new learning materials.

Join Live Lecture

Students can participate in live online lecture sessions hosted by instructors.

Chat During Live Session

Students and instructors can communicate in real time during lectures.

Adaptive Stream (QoE)

The system dynamically adjusts content quality depending on bandwidth conditions and network performance.

Upload and Manage Courses

Instructors can upload, organize, edit, and manage course content.

Host Live Lecture

Instructors can create and conduct live lecture sessions.

View Class Analytics

Instructors can monitor student participation and engagement statistics.

4. Include and Extend Relationships

The diagram also contains include and extend relationships.

a. **Include Relationships**

Include relationships represent mandatory sub-functions that are always required during execution.

Examples include:

- Joining live lectures includes chatting during live sessions.
- Browsing courses may include enrollment functionality.
- Adaptive streaming supports live lecture delivery.

b. **Extend Relationships**

Extend relationships represent optional or conditional behaviors.

For example:

- Downloading offline content extends access to learning materials because downloading is optional.
- These relationships help reduce redundancy and improve diagram organization.

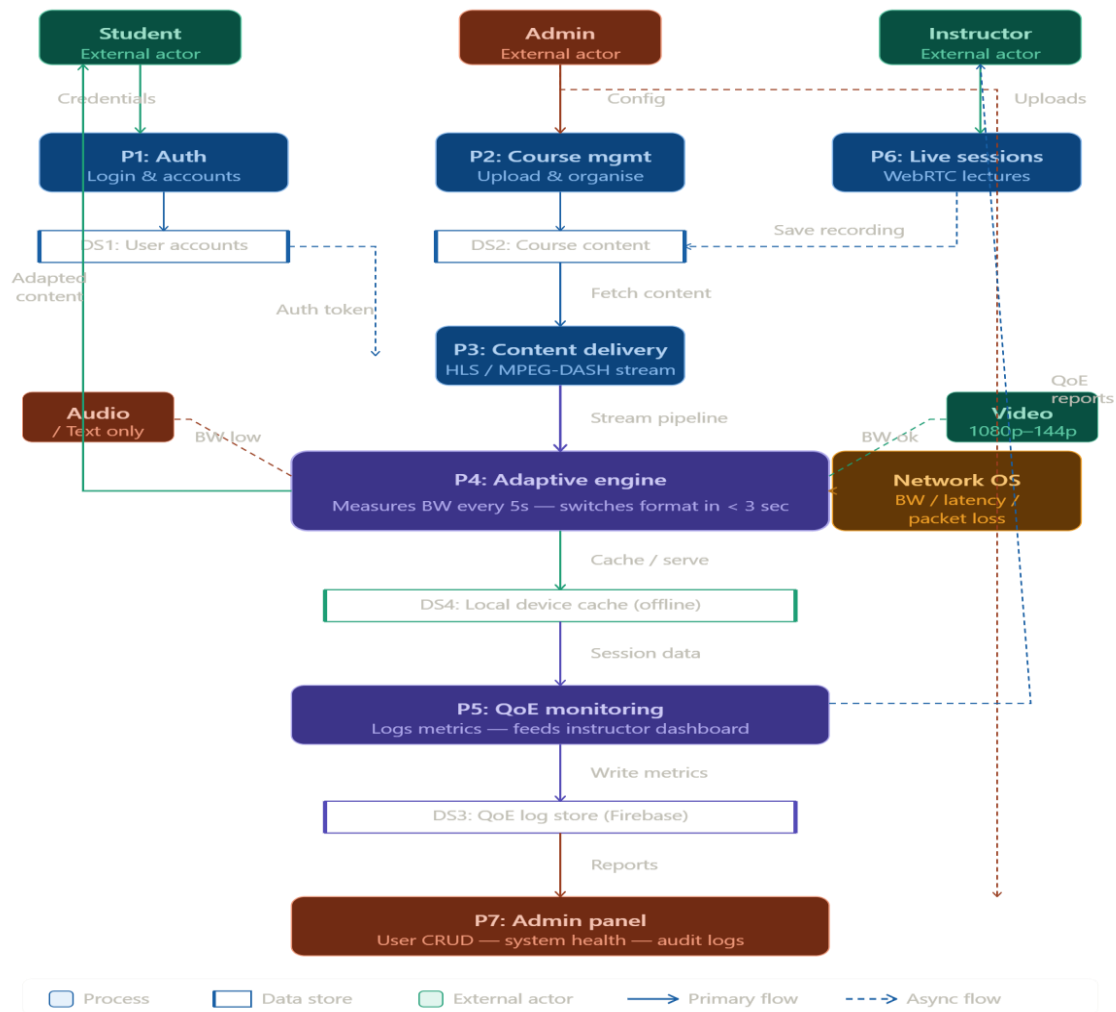
5. **Importance of the Use Case Diagram**

The use case diagram is important because it:

- Captures functional system requirements.
- Shows how users interact with the platform.
- Identifies major system services.
- Helps developers understand user expectations.
- Supports system design and implementation planning.
- Improves communication between technical and non-technical stakeholders.
- The diagram also helps identify access permissions for different user roles.

Data flow Diagram

Shows how information moves through the platform, including processes, external entities, and data stores.



a. External Actors (top row)

There are three external actors or entities that exist outside the system but interact with it directly.

Student sits on the left. The student is the primary consumer of the platform. They log in, request content, and receive whatever format the system decides is best for their current network condition. The arrow going down from the student feeds into P1 (Auth), and the long arrow coming back up on the far left is the system returning adapted content to them.

Admin sits in the middle. The admin does not consume learning content. Their role is purely governance and they manage user accounts, monitor system health, and read audit logs. Their arrow goes down and loops all the way to P7 at the bottom of the diagram.

Instructor sits on the right. The instructor pushes data into the system uploading course materials and starting live sessions. They also receive data back from the system: QoE reports showing how well their students experienced each session. That return arrow is the dashed line running back up the right side from P5.

b. Row 1 [Entry Processes]

These three processes are the first point of contact when any actor interacts with the system.

P1: Auth & Accounts receives the student's credentials (email and password). It verifies them, assigns the correct role (student, instructor, or admin), and issues a session token. This token is then passed downstream shown by the dashed arrow going to P3 and in other to confirm, the user is allowed to access content. Every other process depends on this one working first, which is why it sits at the very top.

P2: Course Management receives uploads from the instructor including videos, notes, quizzes, and downloadable files. It organises them by subject and module, then stores them in DS2. It also handles edits and deletions. Without this process, there is no content for students to access.

P6: Live Sessions handles real-time lectures using WebRTC. The instructor starts a session; students join. There is a live chat feature running alongside the video feed. When the session ends, the recording is automatically saved back into DS2 (shown by the dashed arrow from P6 into DS2) so students can watch it later.

c. Data Stores [First Layer]

DS1: User Accounts sits directly below P1. It is a PostgreSQL database holding every user's registration detail, hashed password, and role. P1 reads and writes here during login and registration.

DS2: Course Content sits below P2. This is cloud storage (AWS S3) holding all videos, audio files, notes, and quiz data uploaded by instructors. Both P2 (on upload) and P6 (on recording save) write into it. P3 reads from it when delivering content to students.

P3: Content Delivery: This process sits in the middle of the diagram and acts as the bridge between stored content and the student's device. It fetches the appropriate material from DS2, checks the auth token from DS1 to confirm the student is enrolled, and then passes the stream down to the adaptive engine. By itself, P3 does not decide what quality to deliver that decision belongs entirely to P4.

P4: Adaptive Engine [The Centrepiece]

This is the most important process on the entire diagram and the core innovation of the platform. Everything above it feeds into it; everything below it flows out of it.

P4 does two things simultaneously. First, it receives the raw content stream from P3. Second, it continuously receives live network measurements from the Network OS the device's own operating system APIs (Android Network Capabilities / iOS NWPathMonitor) reporting bandwidth in kbps, latency in milliseconds, packet loss percentage, and jitter. These measurements arrive every five seconds.

With those two inputs, P4 makes a format decision:

If bandwidth is sufficient, the stream goes to the right branch on Video mode. The engine selects from at least three resolution tiers: 1080p (high quality), 480p (standard), or 144p (low data). It picks the highest tier the current bandwidth can sustain.

If bandwidth drops below the threshold, the stream goes to the left branch which has Audio-only or Text mode. The switch is automatic; the student does not need to do anything. When bandwidth recovers, the engine reverts back to video without user action.

The critical performance requirement written into the diagram is that this entire switch will be detecting the change and applying a new format and must happen in under 3 seconds (NFR-01 from the SRS).

DS4: Local Device Cache: Both branches of the adaptive engine (video and audio/text) feed into DS4 before anything reaches the student's screen. This is local storage on the student's own device and Rooms database on Android, Core Data on iOS. The cache serves two purposes. First, it buffers the stream so playback is smooth even during brief signal dips. Second, it stores downloaded content so students can study completely offline with no internet connection at all. If a student completes a quiz offline, the result is saved here and automatically synced to the server the moment connectivity returns.

P5: QoE Monitoring: After content passes through the cache layer, P5 picks up the session data and begins logging. It records every metric the network OS reported low bandwidth, latency, packet loss, jitter alongside every adaptation event (when did the engine switch quality, what triggered it, how long did the low-quality period last). It also logs user interactions: play, pause, quiz submission, navigation.

All of this is written into DS3: QoE Log Store (Firestore), shown by the downward arrow from P5. The logs are then read back out and compiled into the instructor's QoE dashboard that is the dashed arrow running back up the right side of the diagram to the instructor actor. The instructor can see, per student and per session, exactly what quality of experience was delivered.

DS3: QoE Log Store: This is a Firebase real-time database holding every bandwidth sample, adaptation event, and interaction log generated during sessions. It is the data source for both the instructor dashboard (read up to Instructor) and the admin system health view (read down to P7).

P7: Admin Panel: At the bottom of the diagram, P7 closes the loop for the admin actor. It pulls reports from DS3 to give the admin a system-wide view: active sessions, server load, error logs, and QoE summaries across all users. The admin also uses P7 to perform user CRUD operations: creating, suspending, or deleting student and instructor accounts which write back up into DS1.

The Two Return Arrows: Two long arrows run back up the sides of the diagram and are easy to miss but important to explain:

The left-side arrow (green, solid) carries adapted content back to the student. It originates from P4's output after the format decision and travels all the way back up to the student actor. This represents the final delivery in whatever format the engine chose (video, audio, or text) arriving on the student's device.

The right-side arrow (blue, dashed) carries QoE reports back to the instructor. It originates from P5 and travels up to the instructor actor. This is the feedback loop that lets instructors understand how their students are experiencing the platform in real time.

Reading the Arrow Styles

Solid arrows represent primary, real-time data flows, content moving, credentials being submitted, metrics being pushed. Dashed arrows represent asynchronous or read flows background syncs, report generation, token passing, recording saves. This distinction matters because it shows which parts of the system are time-critical (solid) and which happen in the background (dashed).

Sequence Diagram

a. Introduction

Sequence diagrams are behavioral UML diagrams used to illustrate how different actors and system components interact over time during the execution of system processes. They help visualize the flow of messages, operations, and responses exchanged between users and the system.

For our adaptive e-learning platform, sequence diagrams were developed to model the interactions between users and the platform during major system operations. These

diagrams help explain the logical flow of activities within the system and provide a clear understanding of how the platform functions internally.

The developed sequence diagrams focus on:

Overall system interactions

User authentication processes

Course creation and management

Communication between frontend, backend, and database components

The sequence diagrams also demonstrate how the proposed platform supports online learning activities within low-resource environments.

b. Purpose of the sequence diagrams

The sequence diagrams were designed to achieve the following objectives:

To model the interaction between system actors and components

To describe the chronological order of operations within the platform

To visualize message exchanges between the frontend, backend, and database

To assist in system design and implementation planning

To identify dependencies between platform functionalities

To improve understanding of platform workflow and user interactions

c. Actors and system components

The sequence diagrams involve the following major actors and system components.

Student

The student interacts with the system to:

- Register and log into the platform
- Enroll in courses
- Access learning materials
- Participate in quizzes and assessments
- Track learning progress

Lecturer

- The lecturer interacts with the platform to:
- Log into the system
- Create and manage courses
- Upload learning materials
- Create quizzes and assignments
- Monitor student performance

Administrator

The administrator manages the overall platform by:

- Managing users
- Monitoring system activities
- Configuring system settings

d. App Interface (Frontend)

The Frontend provides the graphical interface through which users interact with the platform. It;

- Receives user inputs
- Displays responses and dashboards
- Sends requests to the backend server
- Displays notifications and system outputs

e. Application Server (Backend)

The application server processes business logic and system operations.

The backend is responsible for:

- User authentication
- Request validation
- Processing platform operations
- Communicating with the database
- Generating system responses

f. Database

The database stores and retrieves platform information including:

- User records
- Course information
- Learning materials
- Quiz results
- Student progress records

g. General system sequence diagram

The general system sequence diagram provides a high-level overview of the major interactions that occur within our platform.

The diagram illustrates how students, lecturers, administrators, Frontend, the application server, and the database interact during key platform activities.

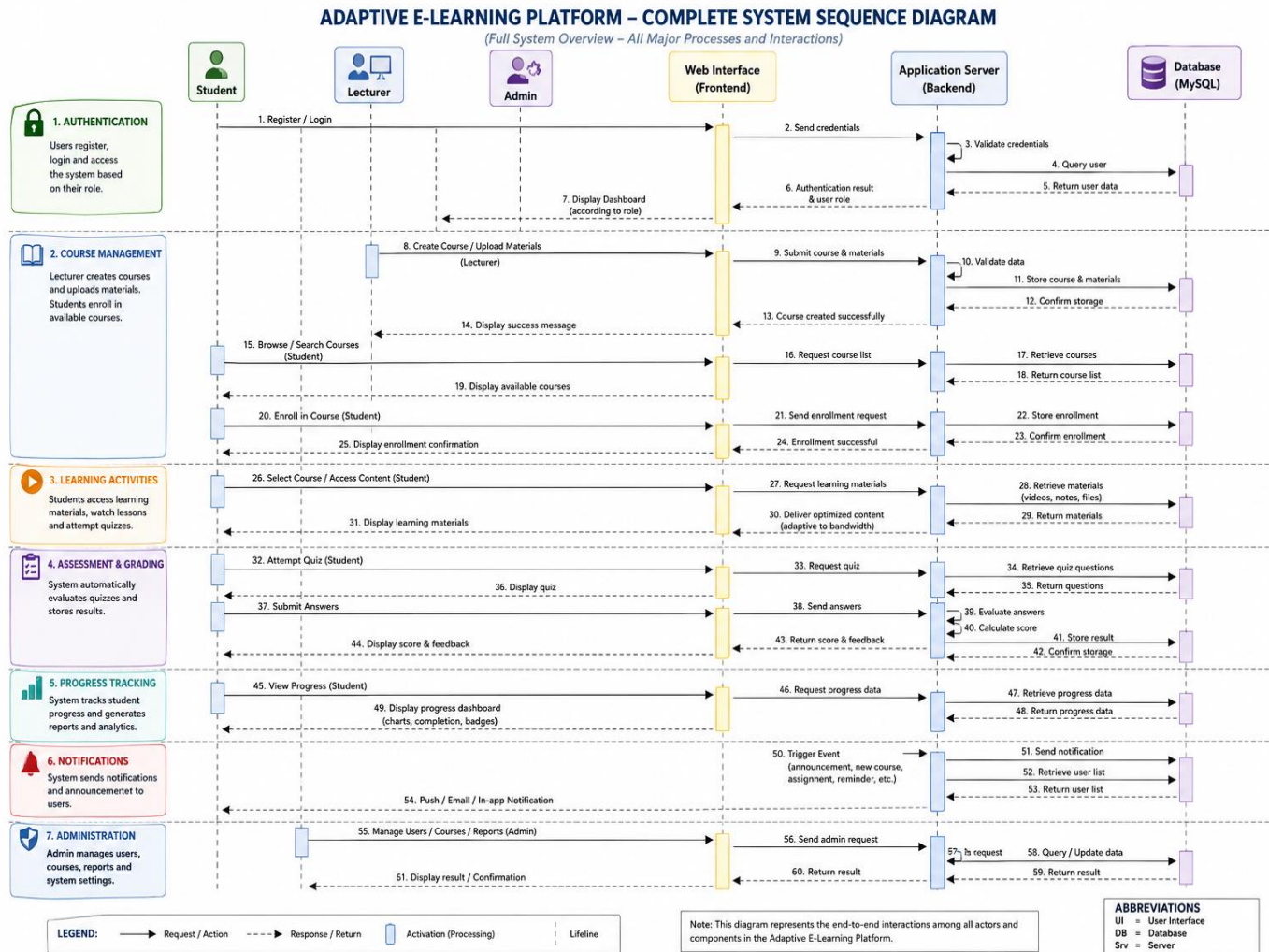
The sequence begins when users access the platform and either register or log into the system. After successful authentication, users are redirected to dashboards based on their roles.

Lecturers can create courses and upload learning materials, while students can browse available courses, enroll in them, and access learning content.

Students may then participate in quizzes and assessments. The application server evaluates submitted answers, calculates scores, stores results in the database, and updates student progress records.

Administrators can manage users, courses, and reports through administrative operations.

The diagram also demonstrates how the system adapts content delivery for low-bandwidth conditions by optimizing learning materials before delivering them to users.



h. Authentication sequence diagram

The authentication sequence diagram models the process of user registration and login within the adaptive e-learning platform.

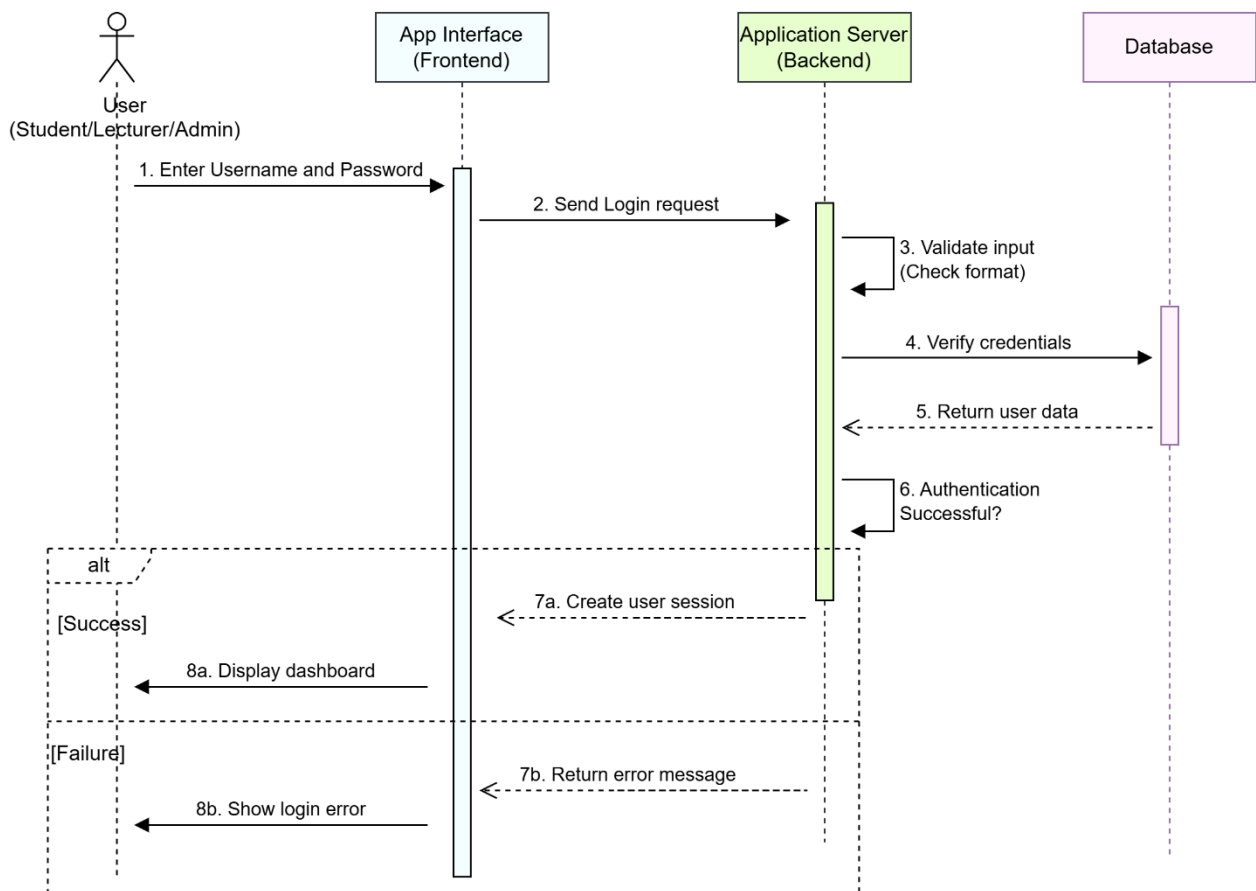
The authentication process begins when a user enters login credentials through the web interface. The frontend sends the credentials to the application server, which validates the input and queries the database to verify user information.

If the credentials are correct, the system authenticates the user and grants access to the appropriate dashboard according to the user role.

If the credentials are invalid, the system displays an authentication error message.

The authentication sequence diagram helps demonstrate the security and validation mechanisms of the platform.

1. Authentication Sequence Diagram.



i. Course creation sequence diagram

The course creation sequence diagram illustrates the interactions involved when a lecturer creates and uploads a course on the platform.

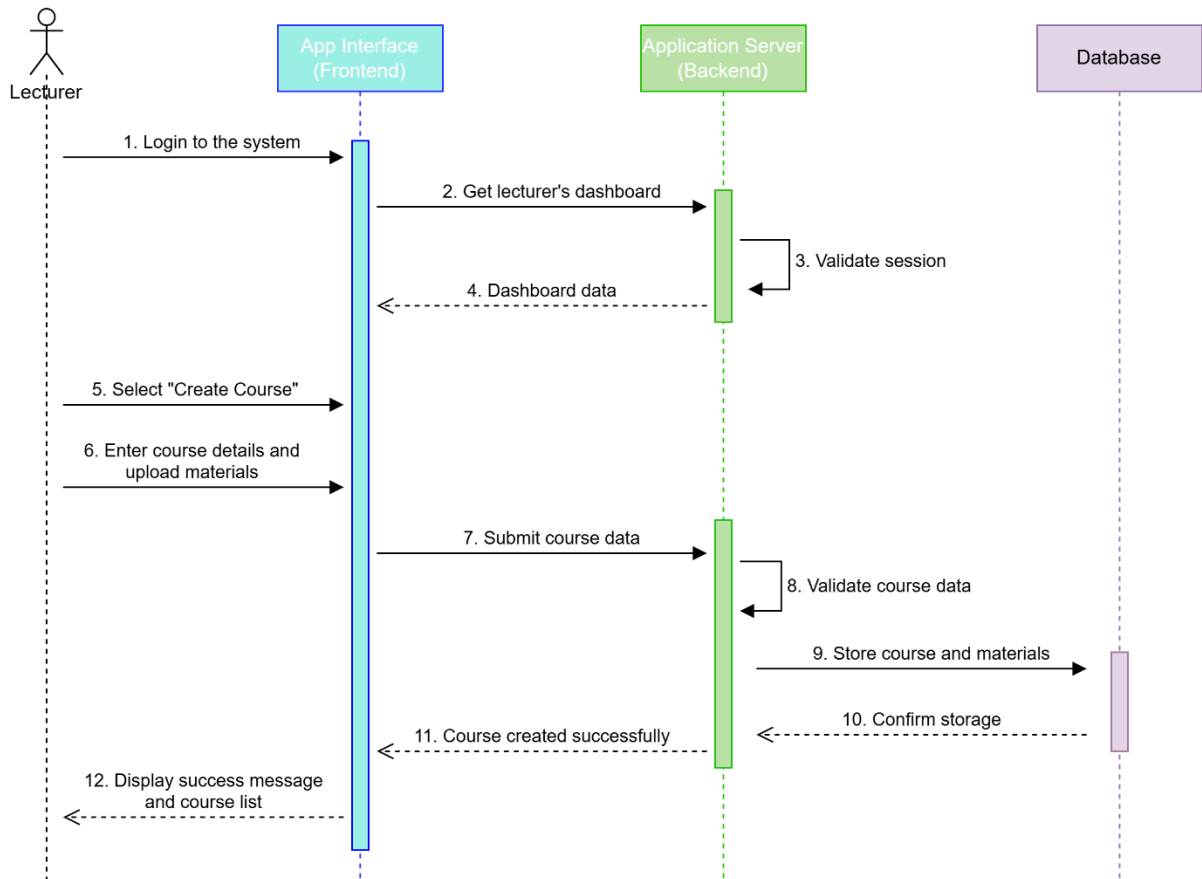
The process begins when the lecturer accesses the course creation interface and enters course details.

The frontend sends the course information to the application server, where the data is validated before being stored in the database.

After successful storage, the system confirms the operation and displays a success message to the lecturer.

This diagram demonstrates the logical interaction flow involved in educational content management.

2. Course Creation Sequence Diagram



j. Conclusion

The sequence diagrams developed for the platform successfully model the interactions between users and system components during major platform operations.

The general system sequence diagram provides a high-level overview of the complete platform workflow, while the authentication and course creation diagrams provide detailed representations of specific system processes.

These diagrams contribute significantly to system analysis, software design, implementation planning, and stakeholder understanding. They also establish a strong technical foundation for the subsequent development phases of our platform.

Class Diagram

1. User Authentication and Account Management

The User Authentication and Account Management Class Diagram represents the subsystem responsible for controlling access to the Adaptive Mobile E-Learning Platform. This subsystem ensures that only authorized users can access platform resources based on their assigned roles and permissions. The design supports multiple user categories including students, instructors, and administrators. The diagram uses inheritance to improve re-usability and reduce redundancy by allowing specialized user classes to inherit common features from a parent User class. The main purpose of this diagram is to model how users interact with the system securely. It defines how user credentials are managed, how authentication is performed, and how user roles are assigned and controlled.

This subsystem supports the following use cases:

- Register User
- Login User
- Reset Password
- Manage Profile
- Assign Roles
- Authenticate User

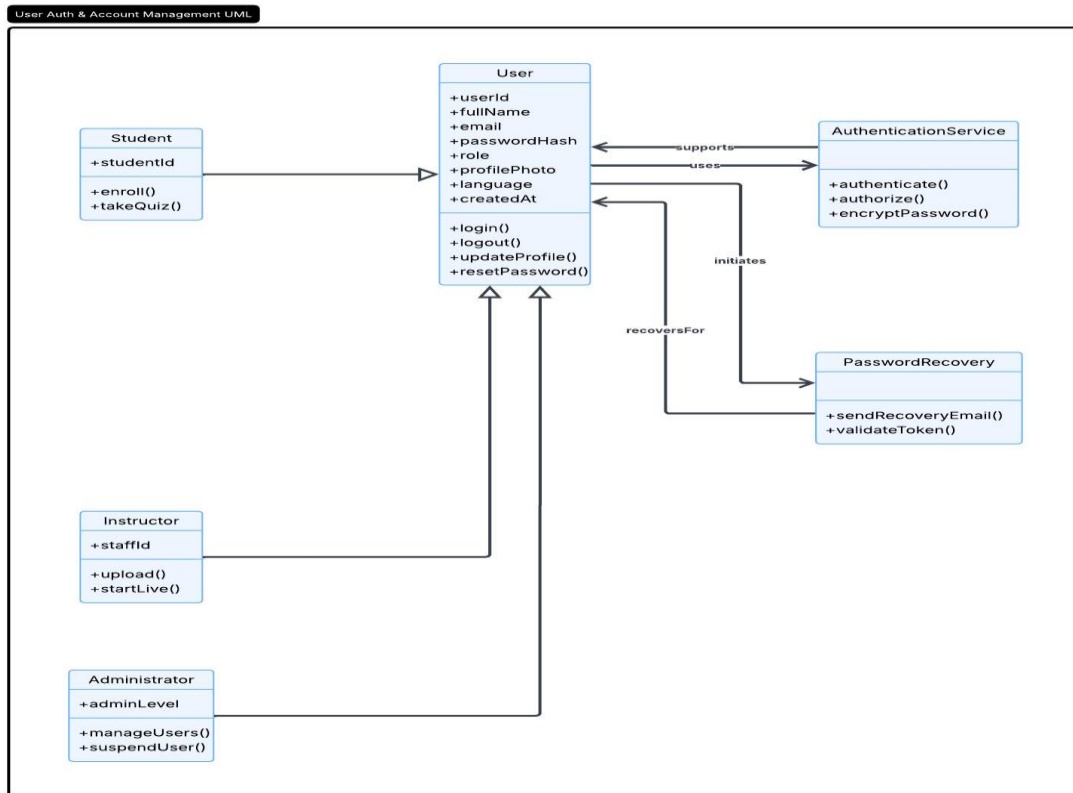
The login() method allows users to access the system after credential verification. The logout() method terminates user sessions to prevent unauthorized access. The updateProfile() method enables users to modify their personal information while resetPassword() allows users to change forgotten passwords securely. The enroll() method allows students to register for available courses while the takeQuiz() method enables participation in assessments and evaluations. This class supports learner interaction with educational resources and academic activities.

The manageUsers() method allows administrators to create, modify, or remove user accounts while suspendUser() restricts access for users who violate platform policies. This class ensures proper system administration and security enforcement.

The authenticate() method validates login credentials while authorize() determines user permissions based on assigned roles. The encryptPassword() method secures passwords before storage in the database. This class improves platform security and protects sensitive user information. The diagram uses inheritance relationships between User and its subclasses including Student, Instructor, and Administrator. Associations also exist between User and AuthenticationService as well as PasswordRecovery. These relationships improve modularity, maintainability, and code reuse.

The User Authentication and Account Management Class Diagram provides a secure and scalable structure for managing users within the Adaptive Mobile E-Learning Platform.

The subsystem ensures secure access control, role management, and user account protection.



2. Course Management Class Diagram

The Course Management Class Diagram models the subsystem responsible for organizing academic courses, modules, and student enrollments within the platform. This subsystem enables instructors to create and manage educational content while allowing students to enroll in courses and access learning materials. The purpose of this diagram is to define the structural organization of courses and learning modules. It also models how students interact with courses through enrollment processes.

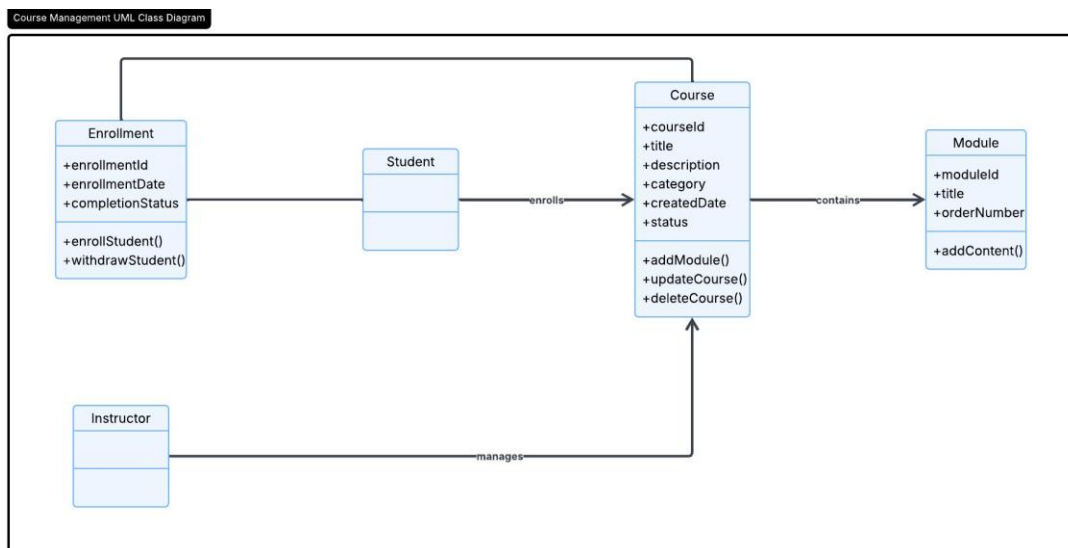
This subsystem supports the following use cases:

- Create Course
- Edit Course
- Delete Course
- Enroll Student
- Search Course
- Organize Modules

The Course class stores information related to educational courses. Attributes such as courseId, title, description, category, and status help identify and organize academic content. The addModule() method allows instructors to attach learning modules to a course. The updateCourse() method modifies course information while deleteCourse() removes obsolete or inactive courses. This class forms the foundation of academic content organization. The Module class represents smaller learning units contained within a course.

The addContent() method enables instructors to attach learning materials such as videos, documents, and quizzes to individual modules. This class improves content organization and modular learning delivery. The Enrollment class manages the relationship between students and courses. The enrollStudent() method registers students into courses while withdrawStudent() removes students from course participation. This class enables effective course registration management.

The diagram contains a one-to-many relationship between Instructor and Course because one instructor may manage several courses. A one-to-many relationship also exists between Course and Module since a course may contain multiple learning modules. A many-to-many relationship exists between Student and Course through the Enrollment class. The Course Management Class Diagram provides a structured framework for organizing educational resources and learner participation within the platform.



3. Learning Content Management Class Diagram

The Learning Content Management Class Diagram represents the subsystem responsible for handling educational resources uploaded into the platform. The subsystem supports multiple content formats including videos, audio lectures, documents, and quizzes. The

purpose of this diagram is to model the creation, storage, management, and offline access of learning materials.

This subsystem supports the following use cases:

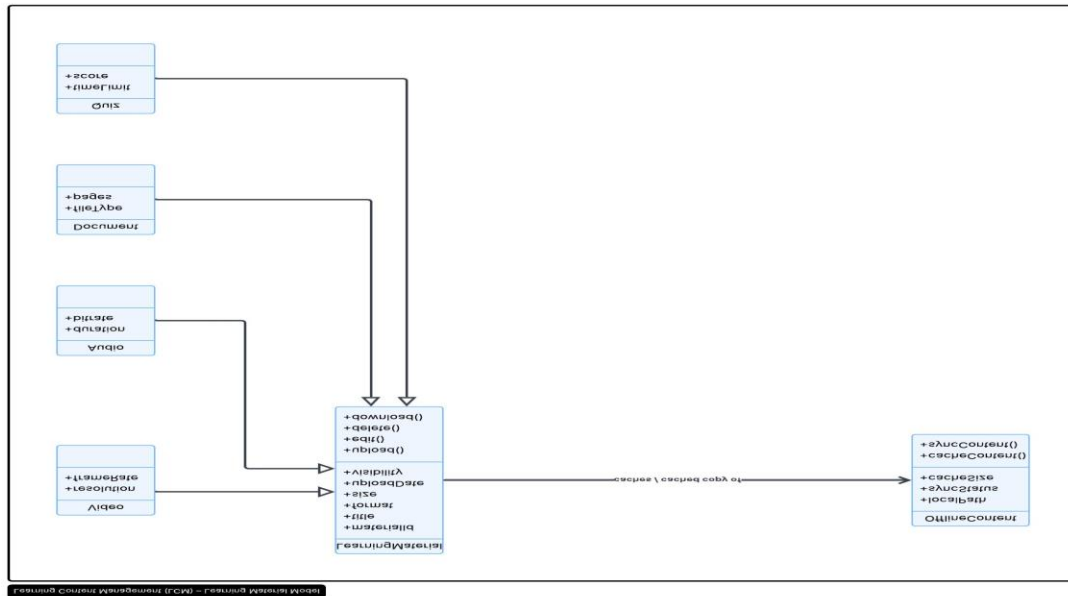
- Upload Materials
- Download Materials
- Edit Materials
- Delete Materials
- Access Offline Content

The LearningMaterial class acts as the parent class for all educational resources. The upload() method allows instructors to add new materials into the system. The edit() method updates existing content while delete() removes outdated resources. The download() method enables students to access learning materials offline. This class centralizes educational content management.

The Video class extends LearningMaterial and represents video-based learning resources. Attributes such as resolution and frameRate support adaptive video delivery and streaming optimization. The Audio class stores audio-based educational materials. The duration and bitrate attributes help optimize audio playback under unstable internet conditions. The Document class handles textual learning resources such as lecture notes and PDFs. The fileType and pages attributes support document organization and accessibility.

The Quiz class represents assessments and evaluations. The timeLimit attribute controls examination duration while score stores student performance results. The OfflineContent class supports offline learning functionality. The cacheContent() method stores materials locally on mobile devices while syncContent() synchronizes offline activities once internet connectivity is restored. This class improves accessibility in low-bandwidth environments.

Inheritance relationships exist between LearningMaterial and the subclasses Video, Audio, Document, and Quiz. Associations also exist between OfflineContent and LearningMaterial. The Learning Content Management subsystem ensures efficient organization and flexible delivery of educational resources across different learning environments.



4. Adaptive Streaming and QoE Management Class Diagram

The Adaptive Streaming and QoE Management Class Diagram models the intelligent subsystem responsible for optimizing learning delivery under unstable network conditions. This subsystem is the core innovation of the Adaptive Mobile E-Learning Platform. The purpose of this subsystem is to monitor user network conditions and dynamically adjust content quality to improve learning continuity and Quality of Experience.

This subsystem supports:

- Adaptive Streaming
- Audio-only Fallback
- Monitor QoE
- Dynamic Resolution Switching
- Detect Network Quality

The QoEMonitor class tracks network performance indicators including latency, packet loss, bandwidth, and buffering rate. The monitorNetwork() method continuously evaluates connectivity conditions while generateMetrics() produces Quality of Experience statistics. This class helps the system make intelligent adaptation decisions.

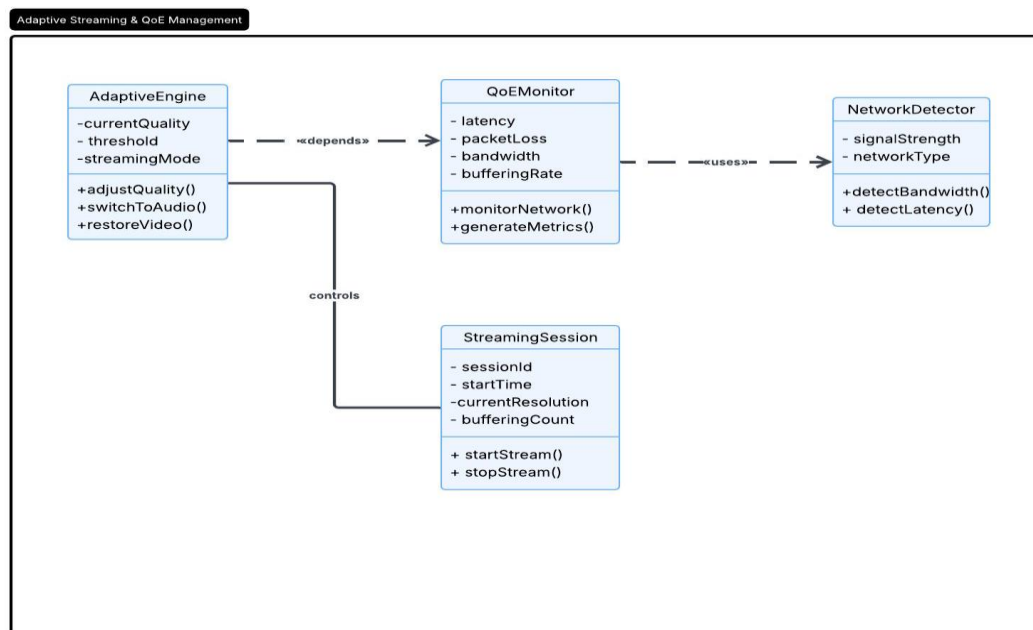
The AdaptiveEngine class controls content adaptation based on network conditions. The adjustQuality() method dynamically changes video quality levels. The switchToAudio() method activates audio-only mode during poor connectivity while restoreVideo() returns to video streaming once network conditions improve. This class minimizes interruptions during online learning.

The StreamingSession class manages active media streaming sessions. The startStream() method initiates content playback while stopStream() terminates active sessions. This class maintains streaming continuity and performance tracking.

The NetworkDetector class measures signal strength and network type. The detectBandwidth() method evaluates internet speed while detectLatency() measures network response delays. This class provides real-time network information for adaptive decisions.

The QoEMonitor class depends on the NetworkDetector class for network measurements. The AdaptiveEngine class relies on QoEMonitor for Quality of Experience data while StreamingSession is controlled by AdaptiveEngine.

The Adaptive Streaming and QoE Management subsystem significantly improves online learning accessibility under unstable internet conditions.



5. Live Lecture Management Class Diagram

The Live Lecture Management Class Diagram models the subsystem responsible for handling real-time teaching and learning activities within the Adaptive Mobile E-Learning Platform. This subsystem enables instructors to conduct synchronous online lectures while allowing students to participate remotely through live streaming and messaging functionalities. The subsystem is designed to support interactive communication, lecture recording, and replay capabilities in order to improve accessibility and continuity of learning.

The purpose of this diagram is to define how live educational sessions are managed within the platform. It illustrates the classes involved in conducting live lectures, exchanging messages during sessions, and storing lecture recordings for future access.

This subsystem supports the following use cases:

- Start Live Lecture
- Join Live Session
- Send Chat Messages
- Record Lectures
- Replay Lectures

The LiveLecture class represents active online lecture sessions conducted by instructors. The attributes lectureId, topic, startTime, endTime, and liveStatus help identify and manage lecture sessions. The lectureId uniquely identifies each lecture while topic stores the lecture subject. The startTime and endTime attributes record the duration of the lecture session, and liveStatus indicates whether the lecture is currently active or ended.

The startLecture() method initializes and activates a live session for students to join. The endLecture() method terminates the lecture session once teaching activities are completed. The recordLecture() method activates the recording functionality to preserve the lecture for future playback. This class serves as the central controller for live educational activities.

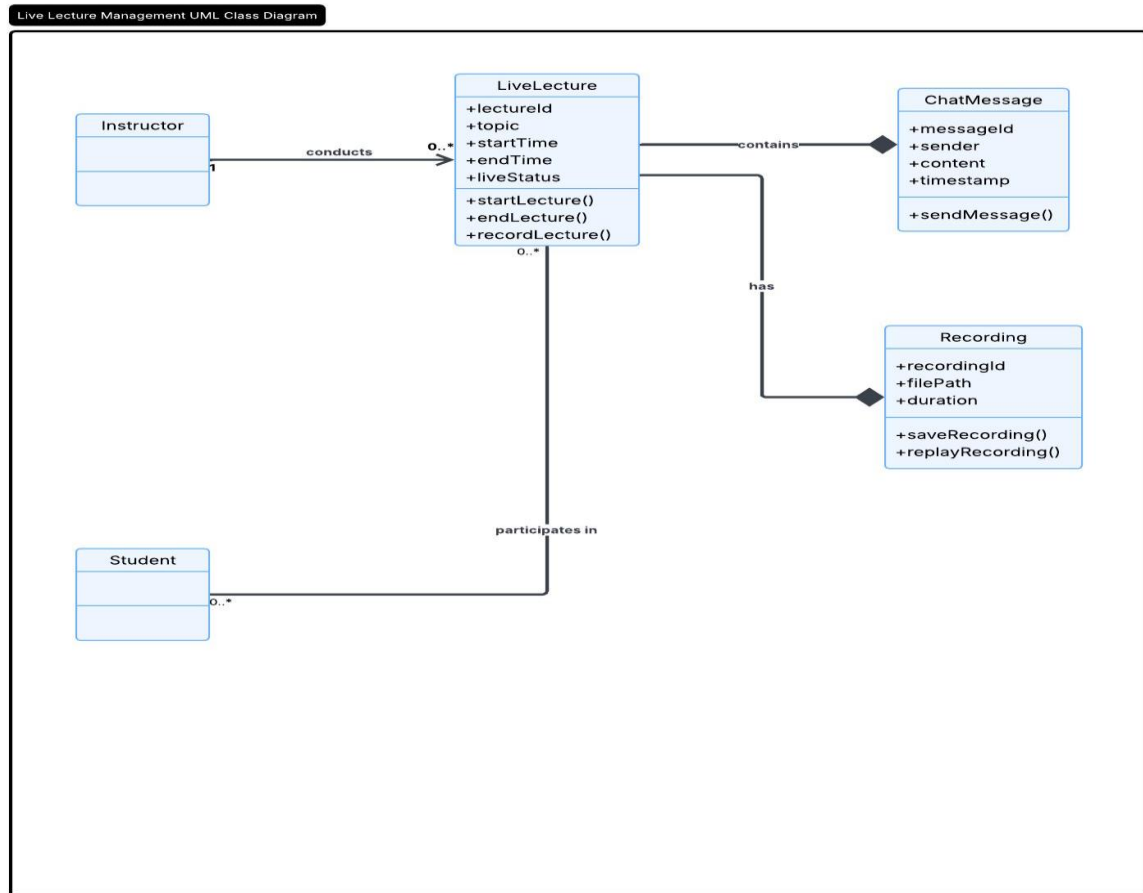
The ChatMessage class manages communication between participants during live sessions. The attributes messageId, sender, content, and timestamp store information related to messages exchanged during the lecture. The sender attribute identifies the participant who sent the message while timestamp records the exact time of communication.

The sendMessage() method allows participants to transmit real-time text messages during lectures. This functionality improves interaction, question-and-answer sessions, and collaborative learning. This class enhances student engagement and communication efficiency.

The Recording class manages lecture recording and playback functionalities. The attributes recordingId, filePath, and duration help identify and store lecture recordings. The filePath attribute specifies the storage location of the recording while duration indicates the total recording time. The saveRecording() method stores lecture recordings permanently within the platform database or cloud storage system. The replayRecording() method allows students to access and replay recorded lectures after the live session has ended. This class supports flexible learning by allowing students to revisit missed or difficult lessons.

A one-to-many relationship exists between Instructor and LiveLecture because one instructor may conduct several live sessions. A one-to-many relationship exists between LiveLecture and ChatMessage since many messages may be exchanged during a single lecture session. A one-to-one relationship exists between LiveLecture and Recording because each lecture session generates a corresponding recording. A many-to-many relationship exists between Student and LiveLecture because multiple students may join multiple live sessions. These relationships ensure efficient management of synchronous learning activities.

The Live Lecture Management subsystem provides real-time online teaching and communication capabilities for the Adaptive Mobile E-Learning Platform. The subsystem supports interactive learning, lecture recording, and replay functionalities that improve accessibility and educational continuity.



6. Student Progress and Analytics Class Diagram

The Student Progress and Analytics Class Diagram models the subsystem responsible for monitoring learner participation, academic performance, and engagement within the platform. This subsystem allows instructors and administrators to evaluate learning outcomes while helping students monitor their academic progress. The purpose of this diagram is to define how learner progress and academic performance are tracked and analyzed.

This subsystem supports the following use cases:

- Track Progress
- Generate Analytics
- Monitor Engagement

- View Dashboard
- Calculate Quiz Scores

The StudentProgress class records and manages learner progress throughout the educational process. The attributes progressId, completionRate, totalTimeSpent, and lastAccessed help track academic activities. The completionRate attribute measures how much course content has been completed while totalTimeSpent records the amount of time spent learning. The updateProgress() method updates learner progress whenever students complete educational activities. The calculateCompletion() method computes the percentage of completed learning tasks. This class supports academic monitoring and personalized learning evaluation.

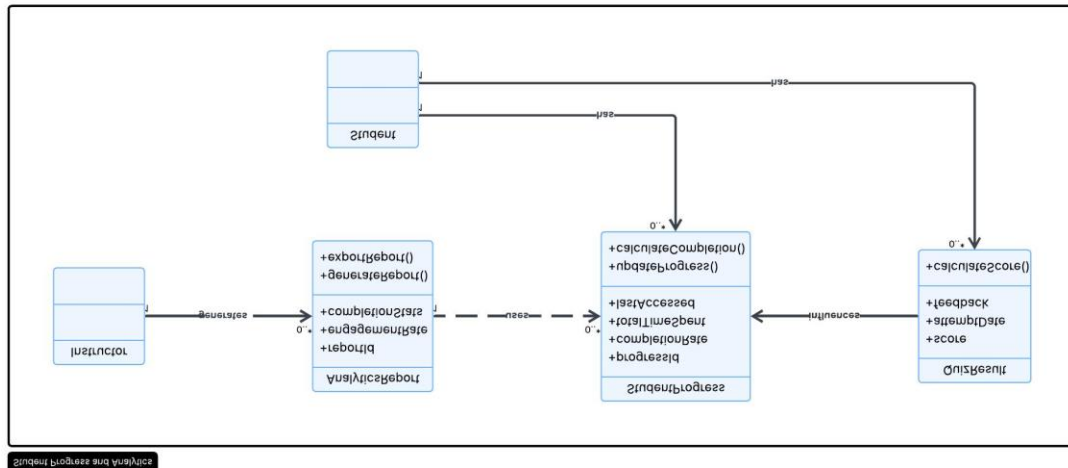
The QuizResult class stores assessment and examination results. The attributes score, attemptDate, and feedback help evaluate learner performance. The score attribute stores the marks obtained during assessments while feedback provides instructor comments and evaluation remarks. The calculateScore() method computes learner performance after quizzes or examinations are completed. This class supports academic assessment and grading activities.

The AnalyticsReport class generates statistical reports related to learner participation and performance. The attributes reportId, engagementRate, and completionStats help summarize educational activities and system usage. The generateReport() method creates analytical reports for instructors and administrators while exportReport() allows reports to be downloaded or shared externally. This class supports educational decision-making and performance evaluation.

A one-to-many relationship exists between Student and StudentProgress because one student may have multiple progress records. A one-to-many relationship also exists between Student and QuizResult since students may attempt multiple quizzes and examinations. A one-to-many relationship exists between Instructor and AnalyticsReport because instructors may generate several reports for different courses.

The AnalyticsReport class depends on StudentProgress to generate learner statistics and performance summaries.

The Student Progress and Analytics subsystem improves educational monitoring by providing accurate learner tracking and performance analysis tools. The subsystem supports instructors in evaluating student engagement and academic achievement.



7. Notification and Communication Class Diagram

The Notification and Communication Class Diagram models the subsystem responsible for information exchange and communication within the Adaptive Mobile E-Learning Platform. The subsystem ensures that students, instructors, and administrators receive timely notifications and announcements regarding academic activities and system updates. The purpose of this subsystem is to improve communication efficiency and ensure that users remain informed about important academic events and activities.

This subsystem supports the following use cases:

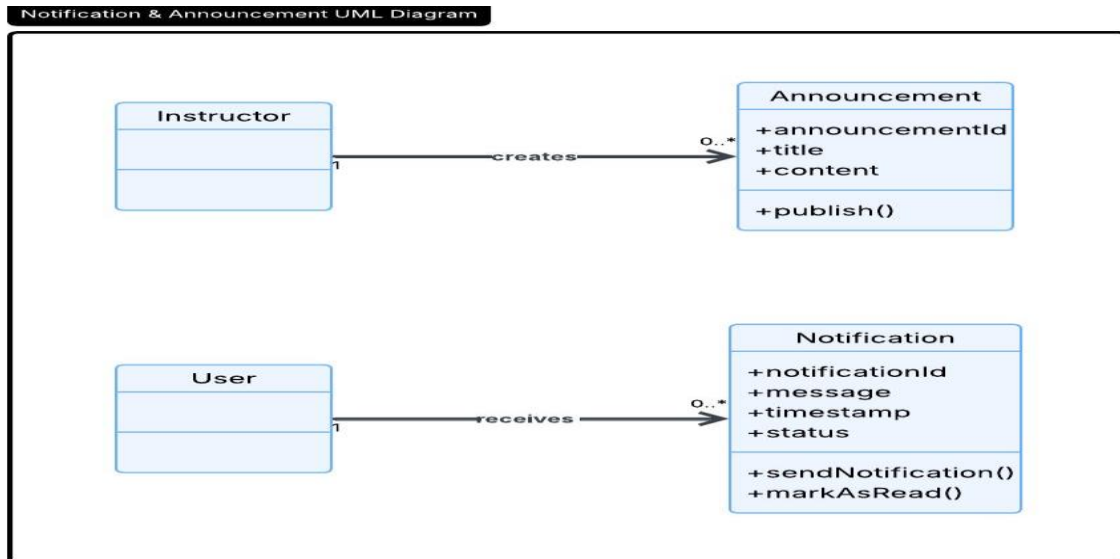
- Send Notifications
- Publish Announcements
- Remind Students
- Notify Uploads

The Notification class manages system-generated alerts and reminders. The attributes `notificationId`, `message`, `timestamp`, and `status` help identify and track notifications. The `status` attribute indicates whether the notification has been read or remains unread. The `sendNotification()` method distributes notifications to users while `markAsRead()` updates notification status after users view the message. This class improves user awareness and interaction with platform activities.

The Announcement class handles official communication from instructors or administrators. The attributes `announcementId`, `title`, and `content` help organize important academic messages. The `publish()` method allows instructors to release announcements related to assignments, schedules, examinations, or course updates. This class supports effective academic communication and coordination.

A one-to-many relationship exists between Instructor and Announcement because instructors may publish several announcements. A one-to-many relationship exists between User and Notification because each user may receive multiple notifications over time. These relationships support efficient information dissemination throughout the platform.

The Notification and Communication subsystem enhances collaboration and information sharing within the platform. It ensures that users receive timely updates and remain informed about important academic activities.



8. Administration and System Monitoring Class Diagram

The Administration and System Monitoring Class Diagram models the subsystem responsible for platform administration, performance monitoring, and system maintenance. This subsystem enables administrators to supervise user activities, monitor server performance, detect technical issues, and maintain platform stability. The purpose of this subsystem is to ensure reliable system operation, administrative control, and technical monitoring of the Adaptive Mobile E-Learning Platform.

This subsystem supports the following use cases:

- Manage Users
- Monitor Performance
- Generate Reports
- Detect Errors
- Suspend Accounts

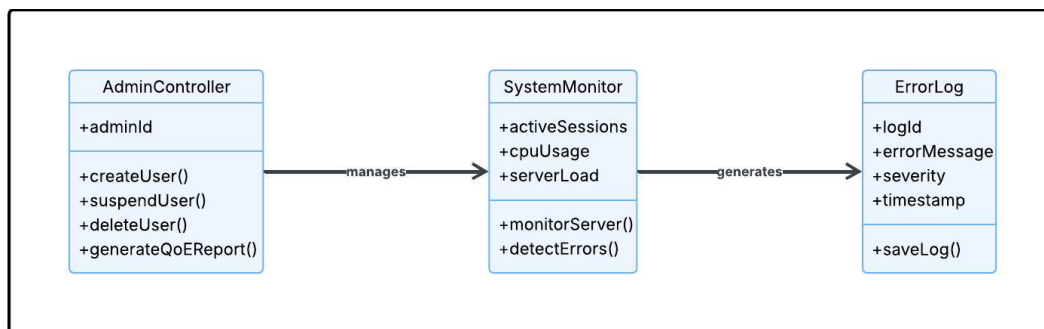
The SystemMonitor class tracks platform performance and operational status. The attributes activeSessions, cpuUsage, and serverLoad help evaluate system health and resource consumption. The monitorServer() method continuously observes server performance and identifies abnormal conditions. The detectErrors() method identifies failures or technical problems occurring within the platform. This class improves system reliability and operational stability.

The ErrorLog class stores information related to technical errors and system failures. The attributes logId, errorMessage, severity, and timestamp help document and categorize detected issues. The saveLog() method records error information into the system database for future troubleshooting and maintenance activities. This class supports technical diagnostics and debugging.

The AdminController class handles administrative operations within the platform. The attribute adminId uniquely identifies administrators. The createUser() method allows administrators to create new user accounts while suspendUser() restricts access for users violating platform policies. The deleteUser() method permanently removes inactive or unauthorized accounts. The generateQoEReport() method creates Quality of Experience reports for evaluating system performance and user satisfaction. This class supports centralized administrative control.

A management relationship exists between AdminController and SystemMonitor because administrators supervise overall platform operations. A one-to-many relationship exists between SystemMonitor and ErrorLog because multiple error logs may be generated during system monitoring activities. These relationships support efficient technical management and maintenance.

The Administration and System Monitoring subsystem ensures proper management, maintenance, and stability of the Adaptive Mobile E-Learning Platform. The subsystem supports technical supervision, user management, and performance evaluation activities.



Deployment Diagram

1. Overview

These UML Deployment Diagrams together describe the complete architecture of an Adaptive E-Learning Platform designed for inclusive online education in low-resource environments. The platform addresses the critical challenge of delivering quality educational content to students in regions with unstable, low-bandwidth internet connectivity by employing a Quality of Experience (QoE) driven adaptive streaming infrastructure.

The system is engineered around a mobile-first philosophy, acknowledging that the majority of learners in underserved areas access educational content via mobile devices. The architecture combines real-time network monitoring, intelligent content adaptation, and multi-format media delivery to ensure learning continuity regardless of network conditions.

2. Diagram 1 System-Level Architecture

Diagram 1 presents the high-level UML Deployment Diagram of the entire platform, illustrating all physical nodes, deployed software components, and the communication paths that connect them. The diagram follows standard UML 3D node notation and clearly delineates client-side devices from backend infrastructure.

2.1 Physical Nodes & Components

The architecture comprises five primary physical nodes:

- a. **Student Mobile Device:** The primary client endpoint, equipped with a Mobile App, Video Player, Audio Module, Local Cache for offline content, and a Network Monitor that measures real-time connectivity quality.
- b. **Instructor Device:** Used by educators to upload course materials, manage live lectures via the Live Lecture Interface, and administer course content through the Instructor Dashboard.
- c. **Internet / Communication Network:** Represented as an unstable low-resource network zone characterised by low bandwidth, high latency, packet loss, and unstable connectivity — reflecting real-world conditions in underserved regions.
- d. **Web / Application Server:** The central backend processing hub hosting the REST API, Authentication Service, Course Management System, Analytics Service, and User Management Module.
- e. **Database Server:** Provides persistent data storage including User, Course, and QoE log databases, as well as Quiz Records and User Activity Logs for analytics and reporting.
- f. **Media / Streaming Server:** Delivers adaptive educational media in multiple formats: HD Video, SD Video, Audio Streams, and Downloadable Learning Materials.

2.2 Communication Paths

The Student Device communicates with both the Application Server and the Media Server via HTTPS/REST through the internet. The Instructor Device connects directly to the Application Server. Internally, the Application Server communicates with the Database Server via JDBC/SQL and coordinates with the Media Server via an internal API to request appropriately-formatted content for each student session.

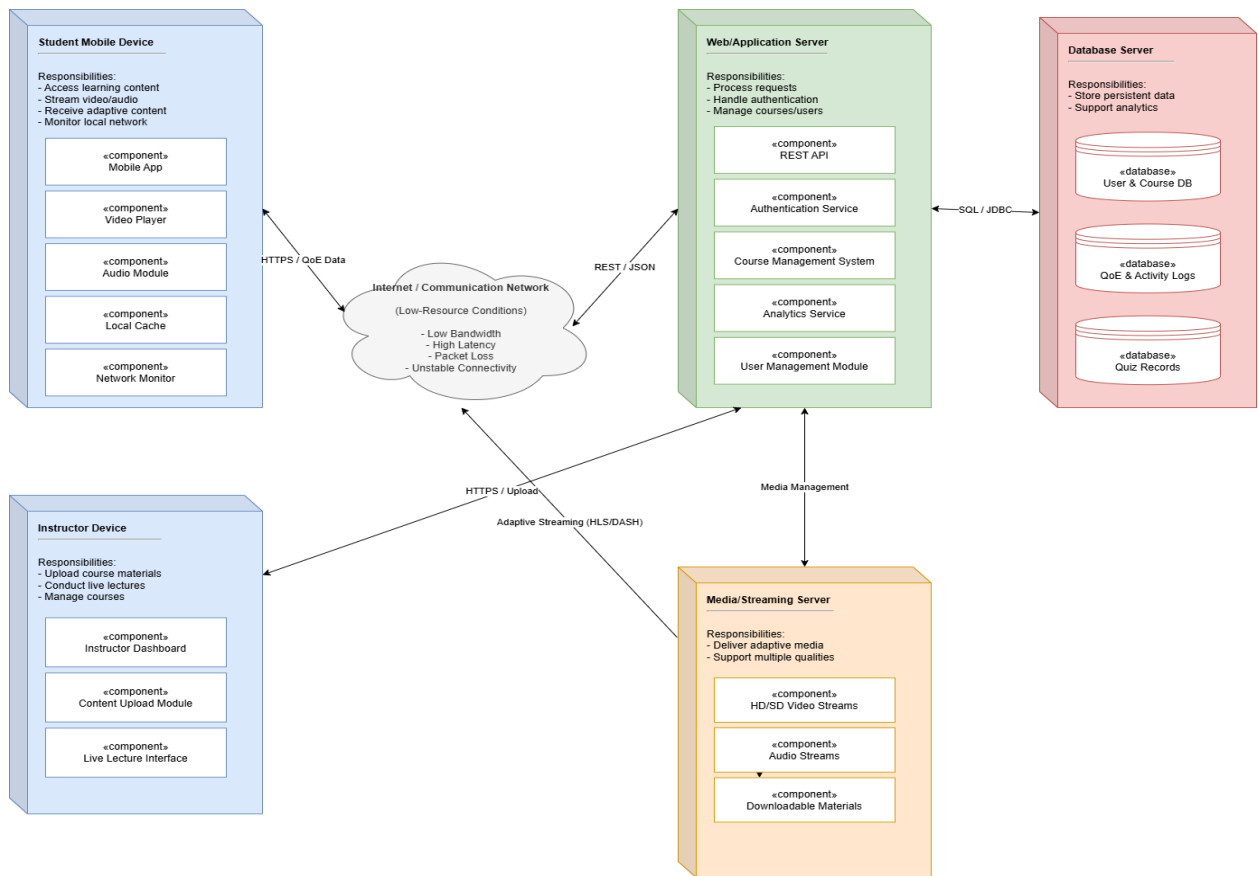


Figure 1: UML Deployment Diagram: Adaptive E-Learning Platform System Architecture

3. Diagram 2:QoE Adaptive Infrastructure

Diagram 2 provides a detailed view of the Quality of Experience (QoE) adaptive subsystem that forms the intelligence layer of the platform. This diagram specifically focuses on the adaptive streaming pipeline: how network conditions are monitored, how adaptation decisions are made, and how media content is delivered in the most appropriate format for each student's current connectivity state.

3.1 QoE Monitoring Module

Deployed alongside the Student Mobile Device, the QoE Monitoring Module continuously evaluates the student's network environment using four specialised components:

- a. **Bandwidth Analyzer:** Measures available downstream bandwidth in real time.
- b. **Latency Detector:** Monitors round-trip delay to application and media servers.
- c. **Packet Loss Monitor:** Tracks the proportion of dropped network packets.
- d. **Network Quality Evaluator:** Aggregates all metrics into a composite QoE score and transmits it to the Adaptation Engine.

3.2 QoE Adaptation Engine

The QoE Adaptation Engine is the decision-making core of the platform. It receives continuous network quality inputs and applies a rule-based content selection framework through four components:

- a. **Adaptation Decision Controller:** Orchestrates the overall adaptation workflow.
- b. **Quality Selection Engine:** Determines the optimal content quality tier for the current session.
- c. **Streaming Adaptation Rules:** A configurable rule set mapping bandwidth thresholds to quality modes.
- d. **Mode Switching Logic:** Executes seamless transitions between media modes without interrupting the student's learning session.

3.3 Adaptive Content States

The engine operates across four defined adaptation states aligned to measurable network thresholds:

Strong Network (≥ 5 Mbps)	HD Video stream delivered, full visual quality for optimal engagement
Moderate Network (1–5 Mbps)	SD Video stream selected, reduced resolution maintains video continuity
Weak Network (< 1 Mbps)	Audio-Only mode engaged, eliminates video overhead, preserving audio instruction
Severe Instability (< 200 Kbps)	Text/Slide content served, offline-capable, ensures zero interruption in learning

3.4 QoE Analytics & Logging Server

All adaptation events, network quality measurements, user experience metrics, and session data are logged to the QoE Analytics and Logging Server. This server maintains four data stores: QoE Logs, Adaptation History, User Experience Metrics, and Session

Monitoring Logs enabling retrospective analysis, platform improvement, and research reporting on inclusive education outcomes in low-resource settings.

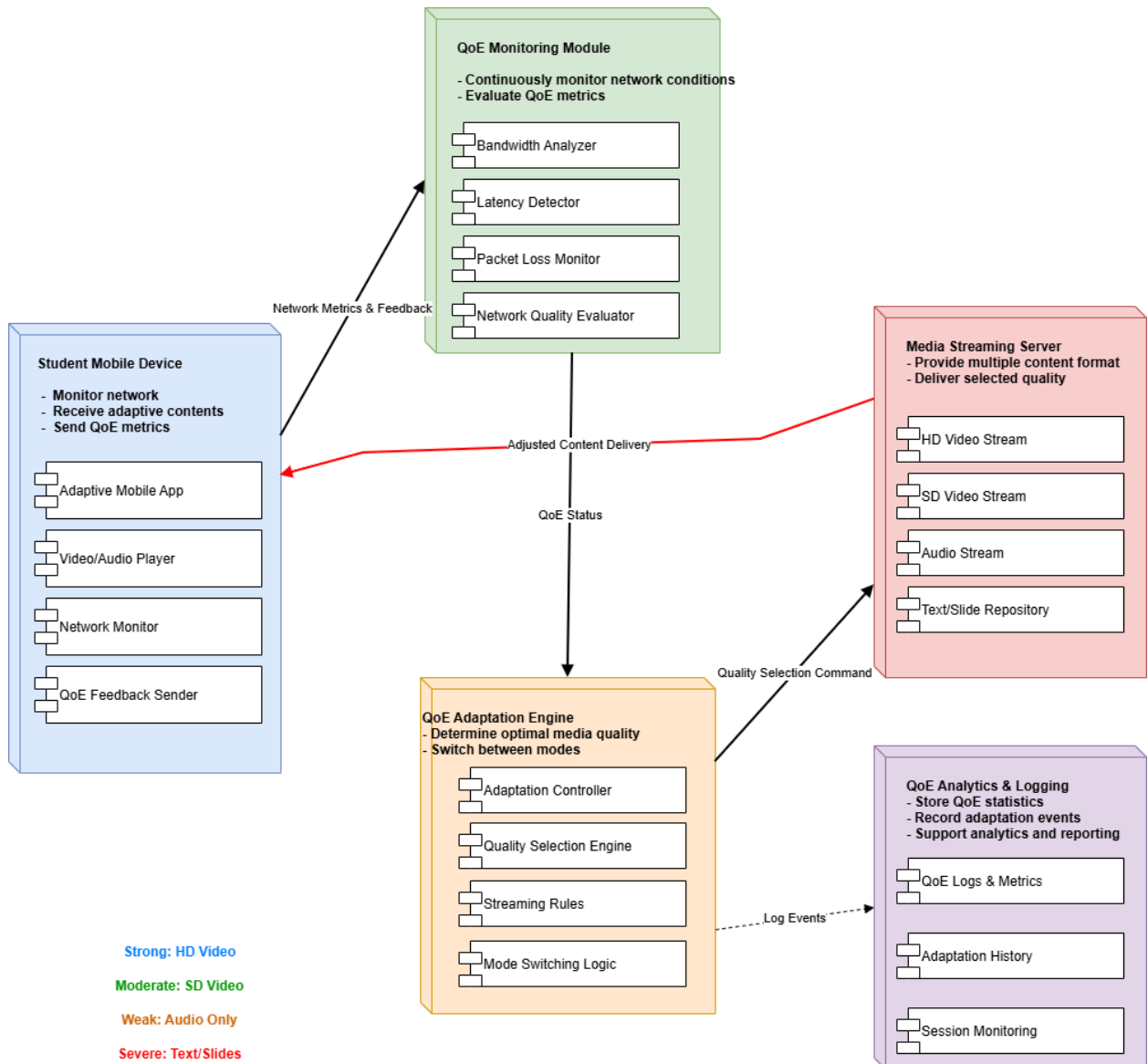


Figure 2 — UML Deployment Diagram: QoE Adaptive Infrastructure & Streaming Control Flow

4. Summary

Together, these two UML Deployment Diagrams define a robust, QoE-aware adaptive e-learning architecture that prioritises learning continuity over network dependency. Diagram 1 establishes the full system topology, encompassing student and instructor devices, server infrastructure, database management, and media delivery while Diagram 2 zooms into the adaptive intelligence that makes the platform viable in low-resource settings. The combination of real-time QoE monitoring, rule-based adaptation, and multi-format content delivery ensures that every student, regardless of connectivity, can access meaningful educational content without disruption.

Overall Conclusion

The system design provides a structured blueprint for developing the adaptive e-learning platform. The various diagrams developed explain the system architecture, workflows, database structure, adaptive QoE mechanisms, and interactions between users and system components. The design emphasizes adaptive content delivery, QoE optimization, and accessibility for low-resource environments with unstable internet connectivity. Overall, the proposed design demonstrates that the system is technically feasible, scalable, and suitable for supporting inclusive online education in low-resource settings.